



Watch the video *Abstract Classes*.

542

TWO TRUTHS & A LIE

Creating and Using Abstract Classes

1. An abstract class is one from which you cannot inherit, but from which you can create concrete objects.
2. Abstract classes usually have one or more empty abstract methods.
3. An abstract method has no body, curly braces, or statements.

The false statement is #1. An abstract class is one from which you cannot create any concrete objects, but from which you can inherit.



You Do It

Creating an Abstract Class

In this section, you create an abstract `Vehicle` class. The class includes fields for the power source, the number of wheels, and the price. `Vehicle` is an abstract class; there will never be a “plain” `Vehicle` object. Later, you will create two subclasses, `Sailboat` and `Bicycle`; these more specific classes include price limits for the vehicle type, as well as different methods for displaying data.

1. Open a new file, and enter the following first few lines to begin creating an abstract `Vehicle` class:


```
public abstract class Vehicle
{
```
2. Declare the data fields that hold the power source, number of wheels, and price. Declare price as protected rather than private, because you want child classes to be able to access the field.


```
private String powerSource;
private int wheels;
protected int price;
```
3. The `Vehicle` constructor accepts three parameters and calls three methods. The first method accepts the `powerSource` parameter, the second accepts

(continues)

(continued)

the `wheels` parameter, and the third method prompts the user for a vehicle price.

```
public Vehicle(String powerSource, int wheels)
{
    setPowerSource(powerSource);
    setWheels(wheels);
    setPrice();
}
```

4. Include the following three get methods that return the values for the data fields:

```
public String getPowerSource()
{
    return powerSource;
}
public int getWheels()
{
    return wheels;
}
public int getPrice()
{
    return price;
}
```

5. Enter the following set methods, which assign values to the `powerSource` and `wheels` fields.

```
public void setPowerSource(String source)
{
    powerSource = source;
}
public void setWheels(int wls)
{
    wheels = wls;
}
```

6. The `setPrice()` method is an abstract method. Each subclass you eventually create that represents different vehicle types will have a unique prompt for the price and a different maximum allowed price. Type the abstract method definition and the closing curly brace for the class:

```
public abstract void setPrice();
}
```

7. Save the file as **Vehicle.java**. At the command prompt, compile the file using the **javac** command.

(continues)

(continued)

Extending an Abstract Class

You just created an abstract class, but you cannot instantiate any objects from this class. Rather, you must extend this class to be able to create any `Vehicle`-related objects. Next, you create a `Sailboat` class that extends the `Vehicle` class. This new class is concrete; that is, you can create actual `Sailboat` class objects.

1. Open a new file, and then type the following, including a header for a `Sailboat` class that extends the `Vehicle` class:

```
import javax.swing.*;
public class Sailboat extends Vehicle
{
```

2. Add the declaration of a length field that is specific to a `Sailboat` by typing the following code:

```
private int length;
```

3. The `Sailboat` constructor must call its parent's constructor and send two arguments to provide values for the `powerSource` and `wheels` values. It also calls the `setLength()` method that prompts the user for and sets the length of the `Sailboat` objects:

```
public Sailboat()
{
    super("wind", 0);
    setLength();
}
```

4. Enter the following `setLength()` and `getLength()` methods, which respectively ask for and return the `Sailboat`'s length:

```
public void setLength()
{
    String entry;
    entry = JOptionPane.showInputDialog
        (null, "Enter sailboat length in feet ");
    length = Integer.parseInt(entry);
}
public int getLength()
{
    return length;
}
```

5. The concrete `Sailboat` class must contain a `setPrice()` method because the method is abstract in the parent class. Assume that a `Sailboat` has a maximum price of \$100,000. Add the following `setPrice()` method that

(continues)

(continued)

prompts the user for the price and forces it to the maximum value if the entered value is too high. Include the `@Override` annotation because the `setPrice()` method overrides the `Vehicle` version.

```
@Override
public void setPrice()
{
    String entry;
    final int MAX = 100000;
    entry = JOptionPane.showInputDialog
        (null, "Enter sailboat price ");
    price = Integer.parseInt(entry);
    if(price > MAX)
        price = MAX;
}
```

6. In Chapter 7, you first used the automatically included `Object` class `toString()` method that converts any object to a `String`. Now, you can override that method for this class by writing your own version as follows. You can include the `@Override` annotation to indicate that this version of `toString()` is intended to override the `Object` class version. When you finish, add a closing curly brace for the class.

```
@Override
public String toString()
{
    return("The " + getLength() +
        " foot sailboat is powered by " +
        getPowerSource() + "; it has " + getWheels() +
        " wheels and costs $" + getPrice());
}
```

7. Save the file as **Sailboat.java**, and then compile the class.

Extending an Abstract Class with a Second Subclass

The `Bicycle` class inherits from `Vehicle`, just as the `Sailboat` class does. Whereas the `Sailboat` class requires a data field to hold the length of the boat, the `Bicycle` class does not. Other differences lie in the content of the `setPrice()` and `toString()` methods.

1. Open a new file, and then type the following first lines of the `Bicycle` class:

```
import javax.swing.*;
public class Bicycle extends Vehicle
{
```

(continues)

(continued)

2. Enter the following `Bicycle` class constructor, which calls the parent constructor, sending it power source and wheel values:

```
public Bicycle()
{
    super("a person", 2);
}
```

3. Enter the following `setPrice()` method that forces a `Bicycle`'s price to be no greater than \$4,000:

```
@Override
public void setPrice()
{
    String entry;
    final int MAX = 4000;
    entry = JOptionPane.showInputDialog
        (null, "Enter bicycle price ");
    price = Integer.parseInt(entry);
    if(price > MAX)
        price = MAX;
}
```

4. Enter the following `toString()` method, and add the closing curly brace for the class:

```
@Override
public String toString()
{
    return("The bicycle is powered by " + getPowerSource() +
        "; it has " + getWheels() + " wheels and costs $" +
        getPrice());
}
```

5. Save the file as **Bicycle.java**, and then compile the class.

Instantiating Objects from Subclasses

Next, you create a program that instantiates concrete objects from each of the two child classes you just created.

1. Open a new file, and then enter the start of the `DemoVehicles` class as follows:

```
import javax.swing.*;
public class DemoVehicles
{
    public static void main(String[] args)
    {
```

(continues)

(continued)

2. Enter the following statements that declare an object of each subclass type.

```
Sailboat aBoat = new Sailboat();  
Bicycle aBike = new Bicycle();
```

3. Enter the following statement to display the contents of the two objects. Add the closing curly braces for the `main()` method and the class:

```
JOptionPane.showMessageDialog(null,  
    "\nVehicle descriptions:\n" +  
    aBoat.toString() + "\n" + aBike.toString());  
    }  
}
```

4. Save the file as **DemoVehicles.java**, and then compile it. After you compile the class with no errors, run this application using the **java DemoVehicles** command. When the application prompts you, enter the length and price for a sailboat, and the price for a bicycle. Figure 11-7 shows output after typical user input.

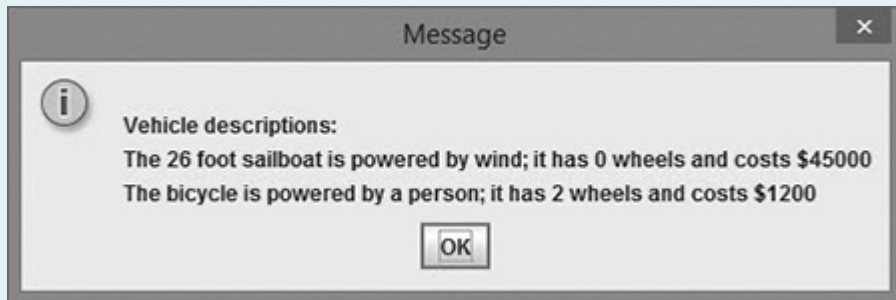


Figure 11-7 Typical output of the DemoVehicles application

Using Dynamic Method Binding

When you create a superclass and one or more subclasses, each object of each subclass “is a” superclass object. Every `SalariedEmployee` “is an” `Employee`; every `Dog` “is an” `Animal`. (The opposite is not true. Superclass objects are not members of any of their subclasses. An `Employee` is not a `SalariedEmployee`. An `Animal` is not a `Dog`.) Because every subclass object “is a” superclass member, you can convert subclass objects to superclass objects.

As you are aware, when a superclass is abstract, you cannot instantiate objects of the superclass; however, you can indirectly create a reference to a superclass abstract object. A reference is not an object, but it points to a memory address. When you create a reference, you do not use the keyword `new` to create a concrete object; instead, you create a variable